

CSE225 Theory Question Bank

Stack & Queue (Array), Stack & Queue (Linked List), Recursion

Source: Lecture 08, Lecture 09 and Lecture 10 slides only.

Total questions: 93, arranged in 7 parts.

Question types: definitions, ADT specifications, write-the-function,

explain-the-logic, time complexity, traces, comparisons,

and condition-based implementation problems.

A model answer follows each question.

How to use this: read the question, write your own answer on paper first, then compare. For code questions, aim to reproduce the structure and the ordering of the pointer statements exactly — that ordering is where most marks are lost.

PART A - Stack: ADT and Array-Based Implementation (Lecture 08)

Q1. Define a stack. State the order it maintains and where insertion and deletion take place.

Answer

A stack is a list in which data items can be added and deleted, and which maintains Last In First Out (LIFO) order. Both insertion and deletion happen at one end only, called the top. The item added most recently is therefore the item removed first.

Q2. Write the complete specification (function, precondition, postcondition) of the Push operation of StackType.

Answer

```
Push(ItemType newItem)
Function      : Adds newItem to the top of the stack.
Precondition  : Stack has been initialized.
Postcondition : If (stack is full), exception FullStack is thrown,
                else newItem is at the top of the stack.
```

Q3. Write the specification of Pop() and Top(). Explain clearly how the two differ.

Answer

```

Pop()
Function      : Removes top item from the stack.
Precondition  : Stack has been initialized.
Postcondition : If (stack is empty), exception EmptyStack is thrown,
                  else top element has been removed from stack.

ItemType Top()
Function      : Returns a copy of the top item on the stack.
Precondition  : Stack has been initialized.
Postcondition : If (stack is empty), exception EmptyStack is thrown,
                  else a copy of the top element is returned.

```

Difference: Pop changes the stack but returns nothing; Top returns the value but leaves the stack unchanged. To both read and remove the top item the client must call Top() and then Pop().

Q4. In the specification of StackType, which items are supplied by the user and which by the ADT? Why is this separation useful?

Answer

Supplied by the user: MAX_ITEMS (the maximum number of items that might be on the stack) and ItemType (the data type of the items). Supplied by the ADT: MakeEmpty, IsEmpty, IsFull, Push, Pop and Top.

The separation is the essence of an ADT: the user decides what is stored and how much, while the ADT guarantees the behaviour of the operations. The user never has to know that the stack is really an array with an integer index.

Q5. Write the header file stacktype.h for the array-based stack.

Answer

```

#ifndef STACKTYPE_H_INCLUDED
#define STACKTYPE_H_INCLUDED

const int MAX_ITEMS = 5;

class FullStack {}; // thrown by Push when stack is full
class EmptyStack {}; // thrown by Pop and Top when stack is empty

template <class ItemType>
class StackType
{
public:
    StackType();
    bool IsFull();
    bool IsEmpty();
    void MakeEmpty();
    void Push(ItemType);
    void Pop();
    ItemType Top();
private:
    int top;
    ItemType items[MAX_ITEMS];
};

#endif // STACKTYPE_H_INCLUDED

```

Q6. Explain the purpose of the classes FullStack and EmptyStack. Why are they declared with empty bodies?

Answer

They are exception classes. FullStack is thrown by Push when the stack has no room; EmptyStack is thrown by Pop and Top when there is nothing to remove or read.

They carry no data because the identity of the type is the whole message. The catch block is selected by the type of the thrown object, so 'catch (FullStack)' already tells the handler exactly what went wrong; no member variables are needed.

Q7. Explain what the include guard (#ifndef / #define / #endif) does and what goes wrong without it.

Answer

The first time the header is included, STACKTYPE_H_INCLUDED is not yet defined, so the compiler defines it and processes the file. Any later #include of the same header finds the macro already defined and skips everything up to #endif.

Without it, a translation unit that includes the header twice (often indirectly) would see the class declared twice, which is a compile-time error.

Q8. Write the constructor, IsEmpty, IsFull and MakeEmpty of the array-based stack, and explain why top starts at -1.

Answer

```
template <class ItemType>
StackType<ItemType>::StackType()      { top = -1; }

template <class ItemType>
bool StackType<ItemType>::IsEmpty()    { return (top == -1); }

template <class ItemType>
bool StackType<ItemType>::IsFull()     { return (top == MAX_ITEMS - 1); }

template <class ItemType>
void StackType<ItemType>::MakeEmpty() { top = -1; }
```

top holds the index of the current top element. Push increments top before storing, so starting at -1 makes the very first item land at items[0]. It also gives IsEmpty a clean test: no valid index equals -1.

Q9. Write Push and Pop for the array-based stack and explain the logic of each line.

Answer

```
template <class ItemType>
void StackType<ItemType>::Push(ItemType newItem)
{
    if (IsFull())
        throw FullStack();
    top++;
    items[top] = newItem;
}

template <class ItemType>
void StackType<ItemType>::Pop()
{
    if (IsEmpty())
        throw EmptyStack();
    top--;
}
```

Push first refuses the operation if there is no room, then moves top to the next free slot and writes the item there. Pop first refuses if there is nothing to remove, then simply moves top back one place.

Q10. Pop() does not erase the popped value from the array. Explain why this is still correct.

Answer

Only the slots from items[0] to items[top] are considered part of the stack. Once top has been decremented, the old value lies above top, so no operation can ever read it: IsEmpty, Top and Pop all work through top. The next Push will overwrite it.

Erasing it would cost time for no benefit, which is why Pop stays $O(1)$.

Q11. Prove that every operation of the array-based stack is $O(1)$.

Answer

Each operation performs a fixed number of steps that does not depend on how many items are currently stored: the constructor and MakeEmpty do one assignment; IsEmpty and IsFull do one comparison; Push does one comparison, one increment and one array write; Pop does one comparison and one decrement; Top does one comparison and one array read.

Because array indexing itself is constant time, none of these involves a loop or a traversal, so all are $O(1)$.

Q12. Why does the array-based StackType have no destructor, while QueType does?

Answer

StackType stores its elements in the member array items[MAX_ITEMS]. That memory is part of the object itself and is reclaimed automatically when the object dies.

QueType allocates its storage with items = new ItemType[maxQue]. Heap memory obtained with new[] is not reclaimed automatically, so ~QueType() must execute delete [] items; to avoid a memory leak.

Q13. A stack has MAX_ITEMS = 5. Trace the value of top through: Push(A), Push(B), Push(C), Pop(), Push(D), Push(E), Push(F), Push(G).

Answer

```
start      top = -1  (empty)
Push(A)    top = 0   items[0] = A
Push(B)    top = 1   items[1] = B
Push(C)    top = 2   items[2] = C
Pop()      top = 1   C is logically removed
Push(D)    top = 2   items[2] = D (overwrites C)
Push(E)    top = 3   items[3] = E
Push(F)    top = 4   items[4] = F -> stack is now full
Push(G)    IsFull() is true -> throw FullStack()
```

Q14. Write a client function that uses StackType to reverse the contents of an array of N characters.

Answer

```
void Reverse(char a[], int n)
{
    StackType<char> s;
    for (int i = 0; i < n; i++)
        s.Push(a[i]);
    for (int i = 0; i < n; i++)
    {
        a[i] = s.Top();
        s.Pop();
    }
}
```

Logic: pushing reads the array left to right, popping returns the items in exactly the opposite order because the stack is LIFO. Both loops run n times with $O(1)$ work each, so the function is $O(N)$. Note MAX_ITEMS must be at least n .

PART B - Applications of Stack (Lecture 08)

Q15. State the algorithm for checking whether a string of parentheses is balanced.

Answer

1. Initialise an empty stack
2. Read next item in the string
 - a) If item is an opening parenthesis, push it into the stack
 - b) Else, if item is a closing parenthesis, pop from stack
3. If there are more items to process, go to step 2
4. Pop the answer off the stack.

Q16. Explain the three possible outcomes of the parenthesis-matching algorithm and what each one means.

Answer

- Empty stack at the end - every opening parenthesis was matched by a closing one: the string is **BALANCED**.
- Non-empty stack at the end - some opening parentheses were never closed: the string is **UNBALANCED**.
- An unsuccessful pop during processing - a closing parenthesis appeared with no opener waiting for it: the string is **UNBALANCED**.

Q17. Why is a stack, rather than a simple counter, the right structure for parenthesis matching? Under what condition would a counter be enough?

Answer

Nesting is inherently last-opened-first-closed, which is exactly the LIFO discipline of a stack. The stack records WHICH opener is still waiting, not merely how many.

With only one kind of bracket, a counter would in fact suffice: increment on '(' and decrement on ')', declaring the string unbalanced if the counter ever goes negative or is non-zero at the end. As soon as several bracket types are involved, e.g. ([]), the counter cannot detect the mismatch but the stack can, because the popped opener can be compared with the closer.

Q18. Trace the algorithm on the string (()) () and show the stack after each item.

Answer

item	action	stack (bottom -> top)

(	push	(
(	push	((
)	pop	(
)	pop	(empty)
(	push	(
)	pop	(empty)

Stack is empty at the end -> the string is **BALANCED**.

Q19. Trace the algorithm on (())) (() and state the verdict.

Answer

item	action	stack

(	push	(
(	push	((
)	pop	(
)	pop	(empty)
)	pop from empty stack	-> UNSUCCESSFUL POP

Verdict: **UNBALANCED**. The fifth character is a closing parenthesis with no opener waiting for it, so the failure is detected before the string ends.

Q20. Write a C++ function, using StackType, that returns true if a string of parentheses is balanced.

Answer

```
bool IsBalanced(char str[], int n)
{
    StackType<char> s;
    for (int i = 0; i < n; i++)
    {
        if (str[i] == '(')
            s.Push(str[i]);
        else if (str[i] == ')')
        {
            if (s.IsEmpty())
                return false;    // unsuccessful pop
            s.Pop();
        }
    }
    return s.IsEmpty();    // leftover openers -> unbalanced
}
```

The IsEmpty test before Pop is what turns the 'unsuccessful pop' case into a clean false instead of an EmptyStack exception. The function is $O(N)$ in the length of the string.

Q21. What is the advantage of postfix notation over infix notation, and why does that advantage exist?

Answer

No parentheses are necessary. In infix notation the operator sits between its operands, so precedence rules and parentheses are needed to say which operator applies first.

In postfix the operator comes after its two operands, so the position in the string already fixes the order of evaluation. There is exactly one way to read the expression, so grouping symbols become redundant.

Q22. Complete the following table.

Answer

Infix	Postfix	Evaluation
2 - 3 * 4 + 5	2 3 4 * - 5 +	-5
(2 - 3) * (4 + 5)	2 3 - 4 5 + *	-9
2 - (3 * 4 + 5)	2 3 4 * 5 + -	-15

Notice that all three infix forms use the same operands in the same order; only the grouping differs, and the postfix form encodes that grouping purely through operator position.

Q23. State the algorithm for evaluating a postfix expression, and explain how validity is detected.

Answer

1. Initialise an empty stack
2. Read next item in the expression
 - a) If item is an operand, push it into the stack
 - b) Else, if item is an operator, pop the top two items off the stack, apply the operator, and push the answer back into the stack
3. If there are more items to process, go to step 2
4. Pop the answer off the stack.

Validity: if exactly one item remains at the end, the expression is valid and that item is the answer. More than one item left, or an unsuccessful pop while applying an operator, indicates an invalid expression.

Q24. Convert $(4+8)*(6-5)/((3-2)*(2+2))$ into postfix form and evaluate it, showing the stack at each step.

Answer

Postfix form (obtained via the binary expression tree): $4\ 8\ +\ 6\ 5\ -\ *\ 3\ 2\ -\ 2\ 2\ +\ *\ /\$

item	action	stack
4	push	4
8	push	4 8
+	pop 8, pop 4, push $4+8=12$	12
6	push	12 6
5	push	12 6 5
-	pop 5, pop 6, push $6-5=1$	12 1
*	pop 1, pop 12, push 12	12
3	push	12 3
2	push	12 3 2
-	pop 2, pop 3, push $3-2=1$	12 1
2	push	12 1 2
2	push	12 1 2 2
+	pop 2, pop 2, push 4	12 1 4
*	pop 4, pop 1, push 4	12 4
/	pop 4, pop 12, push $12/4=3$	3

One item left -> valid expression, answer = 3.

Q25. When an operator is processed during postfix evaluation, why does the order in which the two popped values are used matter?

Answer

The first value popped is the RIGHT operand and the second is the LEFT operand, because the right operand was pushed most recently.

For commutative operators such as + and * the order is harmless, but for - and / it changes the answer: in $6\ 5\ -$, popping gives 5 then 6, and the correct computation is $6 - 5 = 1$, not $5 - 6 = -1$.

Q26. Explain why an expression such as $4\ 8\ +\ +$ is rejected by the evaluation algorithm.

Answer

After $4\ 8\ +$ the stack holds a single value, 12. The second '+' then tries to pop two operands, and the second pop is made on an empty stack. That unsuccessful pop is the signal that the expression is invalid.

Q27. Explain why the expression $4\ 8\ 6\ +$ is rejected.

Answer

Processing pushes 4, 8 and 6, then '+' pops 6 and 8 and pushes 14, leaving the stack as 4 14. The expression ends with two items on the stack instead of one, which indicates an invalid expression - there are too few operators for the operands supplied.

PART C - Queue: ADT and Array-Based (Circular) Implementation (Lecture 08)

Q28. Define a queue. State the order it maintains and where insertion and deletion take place.

Answer

A queue is a list in which data items can be added and deleted, and which maintains First In First Out (FIFO) order. Elements are added to the rear of the queue and removed from the front, so the item that has waited longest is served first.

Q29. Write the specification (function, precondition, postcondition) of Enqueue and Dequeue.**Answer**

```

Enqueue(ItemType newItem)
Function      : Adds newItem to the rear of the queue.
Precondition  : Queue has been initialized.
Postcondition : If (queue is full), FullQueue exception is thrown,
                  else newItem is at rear of queue.

Dequeue(ItemType& item)
Function      : Removes front item from the queue and returns it as item.
Precondition  : Queue has been initialized.
Postcondition : If (queue is empty), EmptyQueue exception is thrown and item is
                  undefined, else front element has been removed from queue and
                  item is a copy of removed element.

```

Q30. Why is Dequeue declared as void Dequeue(ItemType& item) instead of ItemType Dequeue()? What does the reference parameter accomplish?**Answer**

The function has to do two things: remove the front element and hand it back to the caller. Because the return type is void, the value is passed back through the reference parameter: writing to item inside Dequeue writes directly into the caller's variable.

This also lets the postcondition speak sensibly about the failure case - when the queue is empty an exception is thrown and item is simply left undefined.

Q31. A queue is implemented by always inserting at the back of the array and deleting from index 0. Show why deletion is $O(N)$, and state how it is fixed.**Answer**

Removing items[0] leaves a hole at the front. To keep the front element at index 0, every remaining element must be shifted one position to the left. With N elements in the queue that is $N-1$ moves, so deletion is $O(N)$.

The fix is to stop moving the data and instead move the boundary: keep two indices, front and rear, and increment them as elements are dequeued and enqueued. Nothing then has to be shifted, so deletion becomes $O(1)$.

Q32. Explain the 'dead space' problem that arises when front and rear are simply incremented.**Answer**

Each Dequeue does `front++` and each Enqueue does `rear++`, so both indices only ever move to the right. The slots to the left of front hold elements that have already been removed, but front will never return to them.

That region is dead space: unusable memory that grows with every dequeue. Eventually rear reaches the end of the array and the queue reports itself full even though most of the array is free.

Q33. Explain how wrap-around solves the dead space problem, and give the two formulas involved.**Answer**

The array is treated as a circular structure: when an index reaches the end it continues from the beginning, so the dead space at the front is reclaimed. Modulo arithmetic implements this.

```

For Enqueue:  rear = (rear + 1) % maxQue;    // maxQue = array size
For Dequeue:  front = (front + 1) % maxQue;

```

Example with `maxQue = 5`: if `front = 3` and `rear = 4`, then enqueueing 'L' sets `rear = (4+1) % 5 = 0`, so 'L' is stored at index 0, wrapping around to the top of the array.

Q34. Why can the empty state not be distinguished from the full state in a plain circular queue? Illustrate with an example.**Answer**

Both conditions produce the same relationship between front and rear, so the two indices alone cannot tell them apart.

```
maxQue = 5

Case 1 - dequeue down to empty:
[ ][ ][A][ ][ ]   front=2  rear=2
Dequeue(item):
[ ][ ][ ][ ][ ]   front=3  rear=2   <- queue is EMPTY

Case 2 - enqueue up to full:
[C][D][ ][A][B]   front=3  rear=1
Enqueue('E'):
[C][D][E][A][B]   front=3  rear=2   <- queue is FULL

Both end with front=3, rear=2, yet one is empty and the other is full.
```

Q35. Describe the reserved-slot solution to the empty/full ambiguity, and derive the resulting test conditions.**Answer**

Let front indicate the index of the array slot PRECEDING the front element, and never store an item in that slot. One position is therefore always kept empty, which separates the two states.

```
Empty queue when : front == rear
Full queue when : (rear + 1) % maxQue == front
```

Derivation: with the reserved slot, front == rear can only happen when there is no element between them, i.e. the queue is empty. The queue fills up until only the reserved slot is left, which is exactly when advancing rear by one would collide with front. The price of the scheme is one wasted slot, which is why the constructor sets maxQue = max + 1.

Q36. Write the header file queuetype.h for the array-based queue.**Answer**

```
#define DEFAULT_SIZE 500
typedef char ItemType;

class FullQueue {};
class EmptyQueue {};

template <class ItemType>
class QueType
{
public:
    QueType();
    QueType(int max);
    ~QueType();
    void MakeEmpty();
    bool IsEmpty();
    bool IsFull();
    void Enqueue(ItemType newItem);
    void Dequeue(ItemType& item);
private:
    int front;
    int rear;
    ItemType* items;
    int maxQue;
};
```

Q37. Write both constructors, the destructor and MakeEmpty for the array-based queue. Explain why $\text{maxQue} = \text{max} + 1$.

Answer

```

QueueType::QueueType(int max)
{
    maxQue = max + 1;
    front  = maxQue - 1;
    rear   = maxQue - 1;
    items  = new ItemType[maxQue];
}

QueueType::QueueType()
{
    maxQue = DEFAULT_SIZE + 1;
    front  = maxQue - 1;
    rear   = maxQue - 1;
    items  = new ItemType[maxQue];
}

QueueType::~QueueType()    { delete [] items; }

void QueueType::MakeEmpty()
{
    front = maxQue - 1;
    rear  = maxQue - 1;
}

```

$\text{maxQue} = \text{max} + 1$ because one slot is permanently reserved so that the empty and full states can be told apart. The user asked to store max items, so the physical array must be one larger. Setting $\text{front} = \text{rear} = \text{maxQue} - 1$ satisfies $\text{front} == \text{rear}$, i.e. the empty condition.

Q38. Write IsEmpty, IsFull, Enqueue and Dequeue for the circular array queue, and explain the order of statements in each.

Answer

```

bool QueueType::IsEmpty() { return (rear == front); }

bool QueueType::IsFull()  { return ((rear + 1) % maxQue == front); }

void QueueType::Enqueue(ItemType newItem)
{
    if (IsFull())
        throw FullQueue();
    else
    {
        rear = (rear + 1) % maxQue;
        items[rear] = newItem;
    }
}

void QueueType::Dequeue(ItemType& item)
{
    if (IsEmpty())
        throw EmptyQueue();
    else
    {
        front = (front + 1) % maxQue;
        item = items[front];
    }
}

```

In Enqueue, rear points at the last stored element, so it must be advanced to the next free slot before writing. In Dequeue, front points at the slot BEFORE the front element, so it must be advanced first and only then read. Both orders follow directly from what the two indices mean.

Q39. A queue is created as QueType q(4). Trace front, rear and the array contents through Enqueue(A), Enqueue(B), Dequeue, Enqueue(C), Enqueue(D), Enqueue(E).

Answer

maxQue = 5, indices 0..4, start front = 4, rear = 4 (empty)

operation	rear/front update	array	front	rear
start		[][][][][]	4	4
Enqueue(A)	rear=(4+1)%5=0	[A][][][][]	4	0
Enqueue(B)	rear=(0+1)%5=1	[A][B][][][]	4	1
Dequeue(item)	front=(4+1)%5=0, item=A	[A][B][][][]	0	1
Enqueue(C)	rear=(1+1)%5=2	[A][B][C][][]	0	2
Enqueue(D)	rear=(2+1)%5=3	[A][B][C][D][]	0	3
Enqueue(E)	rear=(3+1)%5=4	[A][B][C][D][E]	0	4

Now $(\text{rear}+1)\% \text{maxQue} = (4+1)\%5 = 0 == \text{front}$, so `IsFull()` is true.

Note `items[0]` still physically holds A, but it is the reserved slot and is unreachable; the logical contents of the queue are B, C, D, E.

Q40. Compare the complexity of every operation of the array-based stack and the array-based circular queue.

Answer

Operation	Complexity
StackType constructor	O(1)
IsEmpty / IsFull / MakeEmpty	O(1)
Push / Pop / Top	O(1)
QueType constructors	O(1)
~QueType (delete [] items)	O(1)
MakeEmpty / IsEmpty / IsFull	O(1)
Enqueue / Dequeue	O(1)

Every operation of both structures is O(1) because none of them contains a loop over the stored elements. The circular arithmetic is what keeps Dequeue at O(1) instead of the O(N) of the naive shifting version.

Q41. Given the condition that a client needs to know how many items are currently in a circular queue, write a function that returns that count without changing the queue.

Answer

```
int QueType::Length()
{
    return (rear - front + maxQue) % maxQue;
}
```

Logic: `rear - front` is the number of stored elements when `rear` is ahead of `front`. When the queue has wrapped, that difference is negative, so adding `maxQue` before taking the modulus brings it back into the range `0..maxQue-1`. The empty case `front == rear` correctly yields 0, and the full case gives `maxQue - 1`, which is the true capacity. The function is O(1) and modifies nothing.

PART D - Stack: Linked-List-Based Implementation (Lecture 09)

Q42. What is the limitation of the array-based stack that motivates a linked-list implementation? Explain why using a dynamic array does not remove the limitation.

Answer

With an array, whether static or dynamic, you must anticipate the size of the stack before you use it. If you guess too small the stack overflows; if you guess too large you waste memory.

A dynamic array only moves the guess from compile time to run time - you still have to supply a number when you call new. The linked-list version removes the guess entirely: the stack can grow up to whatever the RAM supports, one node at a time.

Q43. Explain, in the sequence given in the slides, why an array of pointers is rejected and chaining is chosen instead.

Answer

- Each element must be dynamically allocated, e.g. `char* itemPtr = new char;` which places the item somewhere in the free store.
- With many elements there is one pointer per element, so those pointers must be stored somewhere.
- Storing them in an array of pointers fails for the same reason as before: even if that array is itself dynamically allocated, its size still has to be known beforehand.
- The better solution is to chain the elements one after another, each node holding a pointer to the next.
- Then no size need be known in advance; all that is needed is the top pointer.

Q44. Define NodeType and explain the role of each member.

Answer

```
template <class ItemType>
struct NodeType
{
    ItemType info;        // the user's data
    NodeType* next;       // pointer to the next node in the chain
};
```

info stores the value the client asked to keep. next stores the address of the following node, which is what turns a collection of scattered heap cells into an ordered chain. The last node's next is NULL, which marks the end.

Q45. Given 'NodeType<char> node, *nodePtr;', explain the difference between node.info and nodePtr->info, and state what must happen before nodePtr is used.

Answer

node is an actual object, so its member is reached with the dot operator: `node.info = 'D';`. nodePtr is a pointer to an object, so its member is reached with the arrow operator: `nodePtr->info = 'L';`, which is shorthand for `(*nodePtr).info`.

Before the arrow can be used, nodePtr must actually point at a node: `nodePtr = new NodeType<char>;`. Dereferencing an uninitialised pointer is undefined behaviour.

Q46. Write the header file stacktype.h for the linked-list stack, and state the two differences from the array version's interface.

Answer

```

#ifndef STACKTYPE_H_INCLUDED
#define STACKTYPE_H_INCLUDED

class FullStack {};
class EmptyStack {};

template <class ItemType>
class StackType
{
    struct NodeType
    {
        ItemType info;
        NodeType* next;
    };
public:
    StackType();
    ~StackType();
    void Push(ItemType);
    void Pop();
    ItemType Top();
    bool IsEmpty();
    bool IsFull();
private:
    NodeType<ItemType>* topPtr;
};

#endif // STACKTYPE_H_INCLUDED

```

Differences: (1) the linked version adds a destructor, because the nodes are on the heap and must be freed; (2) it drops MakeEmpty, and its only private member is the pointer topPtr instead of an index and an array. MAX_ITEMS also disappears, since there is no fixed capacity.

Q47. Write the constructor, IsEmpty and Top for the linked-list stack and give the complexity of each.

Answer

```

template <class ItemType>
StackType<ItemType>::StackType()    { topPtr = NULL; }           // 0(1)

template <class ItemType>
bool StackType<ItemType>::IsEmpty() { return (topPtr == NULL); } // 0(1)

template <class ItemType>
ItemType StackType<ItemType>::Top()
{
    if (IsEmpty())
        throw EmptyStack();
    else
        return topPtr->info;           // 0(1)
}

```

Q48. Explain how IsFull() is implemented for the linked-list stack and why it is written that way.

Answer

```

template <class ItemType>
bool StackType<ItemType>::IsFull()
{
    NodeType* location;
    try
    {
        location = new NodeType;
        delete location;
        return false;
    }
    catch (std::bad_alloc& exception)
    {
        return true;
    }
}

```

A linked stack has no MAX_ITEMS to compare against, so 'full' can only mean that the free store is exhausted. The function tests this directly by attempting one allocation. If new succeeds, memory is available, so the stack is not full - and the trial node is deleted immediately, since keeping it would be a memory leak. If new fails it throws std::bad_alloc, which is caught and reported as full. The whole thing is O(1).

Q49. Write Push for the linked-list stack and explain each statement.

Answer

```

template <class ItemType>
void StackType<ItemType>::Push(ItemType newItem)
{
    if (IsFull())
        throw FullStack();
    else
    {
        NodeType* location;
        location = new NodeType;           // 1. get a fresh node from the free store
        location->info = newItem;          // 2. store the client's data in it
        location->next = topPtr;           // 3. link the new node to the old top
        topPtr = location;                 // 4. the new node becomes the top
    }
}

```

Steps 3 and 4 are the heart of the operation: the new node is attached to the front of the existing chain, and only then is topPtr moved onto it. No traversal is needed, so Push is O(1).

Q50. What goes wrong if the last two statements of Push are written in the reverse order?

Answer

```

topPtr = location;           // WRONG order
location->next = topPtr;

```

The first line overwrites topPtr, so the address of the previous top node is lost forever. The second line then sets location->next to the value topPtr now holds, which is location itself - the node points at itself.

Result: the stack appears to contain exactly one node, every previously pushed node is leaked (unreachable but still allocated), and following next leads into an infinite self-loop. This is exactly why the link must be established before the top pointer is moved.

Q51. Write Pop for the linked-list stack. Explain why the local pointer tempPtr is necessary.

Answer

```

template <class ItemType>
void StackType<ItemType>::Pop()
{
    if (IsEmpty())
        throw EmptyStack();
    else
    {
        NodeType* tempPtr;
        tempPtr = topPtr;           // 1. remember the node to be removed
        topPtr = topPtr->next;       // 2. move the top down to the next node
        delete tempPtr;             // 3. return the old node to the free store
    }
}

```

tempPtr is needed because the two things that must happen - moving topPtr forward and freeing the old node - conflict. Once topPtr has advanced, the address of the old node is gone and it can never be deleted (a leak). If instead the node is deleted first, topPtr->next would be read from freed memory and the rest of the stack would be lost. Saving the address in tempPtr lets both be done safely.

Q52. Write the destructor of the linked-list stack and justify its complexity.

Answer

```

template <class ItemType>
StackType<ItemType>::~~StackType()
{
    NodeType* tempPtr;
    while (topPtr != NULL)
    {
        tempPtr = topPtr;
        topPtr = topPtr->next;
        delete tempPtr;
    }
}

```

Every node was allocated separately with new, so every node must be deleted separately. The loop body runs once per node, so for N nodes the destructor is O(N). This is the only non-constant-time operation of the linked stack.

Q53. Compare the array-based and linked-list stack implementations across capacity, memory use, the meaning of IsFull, and the complexity of each operation.

Answer

	Array-based	Linked-list-based
Capacity	fixed at MAX_ITEMS	limited only by RAM
Memory	allocated up front, unused slots wasted	one node per element, allocated on demand
Per-element cost	just the item	item + one pointer
Private data	int top, items[]	NodeType* topPtr
IsFull	top == MAX_ITEMS-1	trial new, catch bad_alloc
Destructor	not needed	required, O(N)
Push/Pop/Top	O(1)	O(1)
MakeEmpty	O(1) (top = -1)	not provided in this header

Q54. Given the condition that you may only use the public interface of StackType, write a function that returns the number of items on a stack and leaves the stack unchanged.

Answer

```
template <class ItemType>
int Count(StackType<ItemType> s)      // note: taken BY VALUE
{
    int n = 0;
    while (!s.IsEmpty())
    {
        s.Pop();
        n++;
    }
    return n;
}
```

Logic: the only way to see past the top through the public interface is to pop, so the stack must be destroyed while counting. Passing by value means the caller's stack is untouched and only the local copy is emptied. The function is $O(N)$.

Caution: this relies on the copy having its own nodes. With the header as written there is no copy constructor, so the copy would share nodes with the original and its destructor would free them - a real implementation would need a deep-copy constructor, or the function should push the items back onto the caller's stack using a second temporary stack.

Q55. Given the condition that you may add a member function to the linked-list StackType, write a function that returns the item at the BOTTOM of the stack without modifying it.

Answer

```
template <class ItemType>
ItemType StackType<ItemType>::Bottom()
{
    if (IsEmpty())
        throw EmptyStack();
    NodeType* location = topPtr;
    while (location->next != NULL)
        location = location->next;
    return location->info;
}
```

Logic: the bottom of the stack is the last node in the chain, i.e. the one whose next is NULL, because Push always inserts at the front. A walking pointer starts at topPtr and advances until it finds that node. This is $O(N)$, which is a good illustration of why a stack only promises fast access at the top.

PART E - Queue: Linked-List-Based Implementation (Lecture 09)

Q56. State the advantage of the linked-list queue over the array-based queue, and list which parts of the array implementation become unnecessary.

Answer

With an array you must anticipate the size of the queue beforehand and there is no way to change it during program execution. The linked-list queue grows and shrinks on demand.

Unnecessary in the linked version: maxQue, the modulo wrap-around arithmetic, the reserved slot, and the whole empty/full ambiguity problem. Empty simply becomes front == NULL, and full becomes 'memory exhausted'.

Q57. Write the header file queuetype.h for the linked-list queue.

Answer

```

#ifndef QUEUEUETYPE_H_INCLUDED
#define QUEUEUETYPE_H_INCLUDED

class FullQueue {};
class EmptyQueue {};

template <class ItemType>
class QueType
{
    struct NodeType
    {
        ItemType info;
        NodeType* next;
    };
public:
    QueType();
    ~QueType();
    void MakeEmpty();
    void Enqueue(ItemType);
    void Dequeue(ItemType&);
    bool IsEmpty();
    bool IsFull();
private:
    NodeType<ItemType>* front;
    NodeType<ItemType>* rear;
};

#endif // QUEUEUETYPE_H_INCLUDED

```

Q58. Why does the linked-list queue need TWO pointers when the linked-list stack needs only one?

Answer

A stack inserts and removes at the same end, so one pointer to that end is enough.

A queue inserts at the rear and removes at the front - two different ends. Without a rear pointer, each Enqueue would have to walk the entire chain to find the last node, making insertion $O(N)$. Keeping rear makes both Enqueue and Dequeue $O(1)$.

Q59. Write the constructor, IsEmpty and IsFull for the linked-list queue.

Answer

```

template <class ItemType>
QueueType<ItemType>::QueueType()
{
    front = NULL;
    rear  = NULL;
}

template <class ItemType>
bool QueueType<ItemType>::IsEmpty() { return (front == NULL); }

template <class ItemType>
bool QueueType<ItemType>::IsFull()
{
    NodeType* location;
    try
    {
        location = new NodeType;
        delete location;
        return false;
    }
    catch (std::bad_alloc& exception)
    {
        return true;
    }
}

```

All three are $O(1)$. IsFull uses exactly the same trial-allocation technique as the linked stack, for exactly the same reason.

Q60. Write Enqueue for the linked-list queue and explain the purpose of the 'if (rear == NULL)' test.

Answer

```

template <class ItemType>
void QueueType<ItemType>::Enqueue(ItemType newItem)
{
    if (IsFull())
        throw FullQueue();
    else
    {
        NodeType* newNode;
        newNode = new NodeType;
        newNode->info = newItem;
        newNode->next = NULL;    // it will be the last node
        if (rear == NULL)
            front = newNode;    // queue was empty: it is also the front
        else
            rear->next = newNode; // otherwise link it after the old rear
        rear = newNode;        // in both cases it becomes the new rear
    }
}

```

The test distinguishes the two structurally different situations. If the queue was empty there is no rear->next to write into, and the new node is simultaneously the first and last node, so front must be set. If the queue was non-empty, the old rear node must be linked forward to it. The final rear = newNode is common to both cases, which is why it sits outside the if/else. Enqueue is $O(1)$.

Q61. Write Dequeue for the linked-list queue and explain why 'if (front == NULL) rear = NULL;' is essential.

Answer

```

template <class ItemType>
void QueType<ItemType>::Dequeue(ItemType& item)
{
    if (IsEmpty())
        throw EmptyQueue();
    else
    {
        NodeType* tempPtr;
        tempPtr = front;           // remember the node to remove
        item = front->info;        // copy its data out to the caller
        front = front->next;       // advance the front
        if (front == NULL)        // the queue has just become empty
            rear = NULL;          // so rear must not be left dangling
        delete tempPtr;           // free the removed node
    }
}

```

If the removed node was the only one, front becomes NULL but rear would still hold the address of the node that is about to be deleted - a dangling pointer. The next Enqueue tests rear == NULL to decide whether the queue is empty; with a stale rear that test would fail, and the code would execute rear->next = newNode, writing into freed memory and leaving front pointing nowhere. Resetting rear keeps the two pointers consistent.

Q62. Trace Enqueue('A'), Enqueue('B'), Enqueue('C'), Dequeue, Dequeue, Dequeue on an initially empty linked queue, showing front and rear after each step.

Answer

operation	structure	front	rear
start	(empty)	NULL	NULL
Enqueue(A)	A	-> A	-> A
Enqueue(B)	A -> B	-> A	-> B
Enqueue(C)	A -> B -> C	-> A	-> C
Dequeue	B -> C (item = A)	-> B	-> C
Dequeue	C (item = B)	-> C	-> C
Dequeue	(empty) (item = C)	NULL	NULL

The last Dequeue is the one where front becomes NULL, so rear = NULL is executed and both pointers return to the initial empty state.

Q63. Write MakeEmpty and the destructor for the linked-list queue, and state their complexity. Why is MakeEmpty O(1) for the array queue but O(N) here?

Answer

```

template <class ItemType>
void QueType<ItemType>::MakeEmpty()
{
    NodeType* tempPtr;
    while (front != NULL)
    {
        tempPtr = front;
        front = front->next;
        delete tempPtr;
    }
    rear = NULL;
}

template <class ItemType>
QueType<ItemType>::~~QueType() { MakeEmpty(); }

```

Both are O(N). The array queue keeps all its elements inside one block, so emptying it is just front = rear = maxQue - 1, a constant-time index reset; the memory is released later by a single delete []. The linked queue holds N separately allocated nodes, and each one must be individually returned to the

free store, so the work is proportional to N.

Q64. Compare all operations of the array-based and linked-list queues by complexity, and explain every place where they differ.

Answer

Operation	Array (circular)	Linked list
Constructor	$O(1)$	$O(1)$
IsEmpty	$O(1)$	$O(1)$
IsFull	$O(1)$	$O(1)$
Enqueue	$O(1)$	$O(1)$
Dequeue	$O(1)$	$O(1)$
MakeEmpty	$O(1)$	$O(N)$
Destructor	$O(1)$	$O(N)$

The only differences are MakeEmpty and the destructor, and both have the same cause: bulk deallocation of one array block versus node-by-node deallocation. In exchange, the linked version has no capacity limit and wastes no unused slots.

Q65. Given the condition that a client wants to inspect the front element without removing it, add a member function Front() to the linked-list queue.

Answer

```
template <class ItemType>
ItemType Queue<ItemType>::Front()
{
    if (IsEmpty())
        throw EmptyQueue();
    return front->info;
}
```

Logic: this is the queue's counterpart of the stack's Top(). The front pointer already addresses the element to be served next, so the value is read directly with no traversal and no deletion, making the operation $O(1)$. The empty check is required for the same reason as in Dequeue.

PART F - Recursion: Concepts and the Run-Time Stack (Lecture 10)

Q66. Define recursion. State one advantage and one disadvantage.

Answer

Recursion is a technique that solves a problem by solving smaller versions of the same problem. In a program this becomes a function that calls itself.

Advantage: recursive algorithms can simplify the solution of a problem, often resulting in shorter, more easily understood source code. Disadvantage: they are often less efficient than non-recursive (iterative) solutions in both time and space, because of the overhead of the function calls.

Q67. Compare recursion and iteration under four headings: the control structure used, how repetition is achieved, how it terminates, and how repetition is controlled.

Answer

	Iteration	Recursion
-----	-----	-----
Structure used	repetition structures (for, while, do...while)	selection structures (if, if...else, switch)
Repetition via	explicit use of a repetition structure	repeated function calls
Terminates when	loop-continuation condition fails	the base case is satisfied
Repetition controlled by	by using a counter	by dividing the problem into simpler one(s)

Q68. Write the general form of a recursive function and explain each part.

Answer

```
Solve (Problem)
{
    if (Problem is minimal / not decomposable: a base case)
        solve Problem directly; i.e., without recursion
    else {
        (1) Decompose Problem into one or more similar,
            strictly smaller subproblems: SP1, SP2, ..., SPn
        (2) Recursively call Solve on each subproblem:
            Solve(SP1), Solve(SP2), ..., Solve(SPn)
        (3) Combine the results into a solution of Problem
    }
}
```

The word 'strictly smaller' is what guarantees termination: since each call is on a smaller instance and the base case handles the smallest one, the chain of calls must eventually reach it.

Q69. Give both the recursive and the closed-form definitions of $n!$, and write the recursive C++ function.

Answer

Recursive:	$n! = 1$	if $n = 0$
	$n! = (n-1)! * n$	if $n > 0$
Closed form:	$n! = 1$	if $n = 0$
	$n! = 1*2*3*...*(n-1)*n$	if $n > 0$

```
int Factorial(int n)
{
    if (n == 0)                // base case
        return 1;
    else
        return n * Factorial(n-1); // decompose, solve, combine
}
```

The recursive definition describes $n!$ in terms of a smaller factorial; the closed form gives the answer directly without referring to $n!$ again.

Q70. Trace Factorial(5), showing both the descent and the return of values.

Answer

Descent (calls going down)	Return (values coming back)
-----	-----
5!	5! = 5 * 24 = 120 is returned
5 * 4!	4! = 4 * 6 = 24 is returned
4 * 3!	3! = 3 * 2 = 6 is returned
3 * 2!	2! = 2 * 1 = 2 is returned
2 * 1!	1! = 1 * 1 = 1 is returned
1 * 0!	1 is returned (base case)
1	
Final value = 120	

Nothing is multiplied on the way down; the multiplications only happen as the calls return, which is why the run-time stack must hold all five pending calls at once.

Q71. How does the computer implement recursion? Explain activation records.

Answer

It uses a stack called the run-time stack to keep track of the function calls. Each time a function is called - recursively or not - an activation record is created and pushed onto that stack. When the call returns, the corresponding activation record is popped.

The activation record stores everything the suspended call will need when control comes back to it: its parameters, its local variables, and the place at which to resume execution. A stack is the correct structure because calls finish in exactly the reverse of the order they were made (LIFO).

Q72. Describe, step by step, what happens when function b executes $z = a(y)$, where a is `int a(int w)`.

Answer

- The system stops executing function b and starts executing function a.
- Since it must come back to b later, it stores everything about b that will be needed - x, y, z, and the place to start executing upon return - in an activation record pushed onto the run-time stack.
- The argument passed from b is bound to the parameter w of a.
- Control is transferred to function a.
- After a finishes, the activation record is popped, all the old values of b's parameters and variables are restored, and a's return value replaces $a(x)$ in the assignment statement.

There is no difference at all between recursive and non-recursive calls in this mechanism; a recursive call is simply a call whose target happens to be the same function.

Q73. For `int f(int x){ int y; if(x==0) return 1; else { y = 2*f(x-1); return y+1; } }`, trace f(3) using the run-time stack.

Answer

```

Push copies going down:
f(3): x=3, y=?  needs 2*f(2)  -> call f(2)
f(2): x=2, y=?  needs 2*f(1)  -> call f(1)
f(1): x=1, y=?  needs 2*f(0)  -> call f(0)
f(0): x=0      base case, return 1

Pop copies coming back:
f(0) = 1
f(1): y = 2*1 = 2,    return y+1 = 3
f(2): y = 2*3 = 6,    return y+1 = 7
f(3): y = 2*7 = 14,   return y+1 = 15

Value returned by the call is 15.

```

Four activation records exist simultaneously at the deepest point, and they are popped in the reverse order of their creation.

Q74. State the four steps for writing a recursive function, and apply them to Factorial.**Answer**

- Determine the size factor.
- Determine the base case(s) - the one(s) for which you know the answer.
- Determine the general case(s) - the one(s) where the problem is expressed as a smaller version of itself.
- Verify the algorithm.

For Factorial: the size factor is n ; the base case is $n == 0$, whose answer is 1; the general case is $n * \text{Factorial}(n-1)$; and verification is done with the three-question method below.

Q75. State the Three-Question Verification Method and apply all three questions to Factorial.**Answer**

- The Base-Case Question: is there a non-recursive way out of the function, and does the routine work correctly for this base case?
- The Smaller-Caller Question: does each recursive call to the function involve a smaller case of the original problem, leading inescapably to the base case?
- The General-Case Question: assuming that the recursive call(s) work correctly, does the whole function work correctly?

Applied to Factorial: (1) yes - $n == 0$ returns 1 without recursion, and $0!$ is indeed 1. (2) yes - the call is $\text{Factorial}(n-1)$, so n decreases by exactly 1 each time and, for $n \geq 0$, must reach 0. (3) yes - assuming $\text{Factorial}(n-1)$ correctly returns $(n-1)!$, then $n * (n-1)!$ is $n!$ by definition.

Q76. State the three criteria given in the slides for deciding whether to use a recursive solution.**Answer**

- The recursive version is shorter and simpler than the non-recursive solution.
- The depth of recursive calls is relatively shallow.
- The recursive version does about the same amount of work as the non-recursive version.

If any of these fails - especially the third - the iterative version is usually preferable, since recursion could be very slow because of the extra memory and time overhead due to function calls.

Q77. Using $\text{Comb}(6,4)$ as the example, explain why plain recursion can be very inefficient, and describe the remedy.**Answer**

The call tree for $\text{Comb}(6,4)$ branches into $\text{Comb}(5,3)$ and $\text{Comb}(5,4)$, each of which branches again. Many of the resulting subproblems are identical - for example $\text{Comb}(4,3)$ appears under both branches - and plain recursion recomputes each occurrence from scratch. The number of calls grows explosively even though the number of DISTINCT subproblems is small.

The remedy is dynamic programming, which is mainly an optimization over plain recursion: the idea is to simply store the results of subproblems, so that they do not have to be re-computed when needed later. It is studied in CSE 373.

Q78. Explain why recursion consumes more memory than iteration, even when both perform the same number of operations.**Answer**

An iterative loop reuses one set of local variables for every pass. A recursive function creates a new activation record for every pending call, and those records all stay alive on the run-time stack until the calls return.

A recursion of depth d therefore needs $O(d)$ stack space in addition to the actual work, and if d grows with the input the stack can be exhausted. This is the space half of the statement that recursion is often less efficient in both time and space.

PART G - Recursion Applied: Binary Search and Sorted Linked Lists (Lecture 10)

Q79. Write the iterative version of RetrieveItem (binary search) and identify what each of its variables does.

Answer

```
template <class ItemType>
void SortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    int midPoint;
    int first = 0;
    int last = length - 1;
    found = false;
    while ((first <= last) && !found)
    {
        midPoint = (first + last) / 2;
        if (item < info[midPoint])
            last = midPoint - 1;
        else if (item > info[midPoint])
            first = midPoint + 1;
        else
        {
            found = true;
            item = info[midPoint];
        }
    }
}
```

first and last delimit the part of the array still under consideration; midPoint is the element compared on this pass; found is both the result and the loop's early-exit flag. The loop ends either when the item is found or when the range becomes empty (first > last).

Q80. For recursive binary search, state the size factor, the base cases and the general cases.

Answer

Size factor: the number of elements in (info[first] ... info[last]).

Base cases: (1) if first > last, return false; (2) if item == info[midPoint], return true.

General cases: if item < info[midPoint], search the first half; if item > info[midPoint], search the second half.

Q81. Write the recursive BinarySearch function together with the RetrieveItem wrapper.

Answer

```

template <class ItemType>
void SortedType<ItemType>::RetrieveItem(ItemType& item, bool& found)
{
    found = BinarySearch(info, item, 0, length-1);
}

template <class ItemType>
bool BinarySearch(ItemType info[], ItemType& item, int first, int last)
{
    int midPoint;
    if (first > last)                                // base case 1
        return false;
    else {
        midPoint = (first + last) / 2;
        if (item < info[midPoint])
            return BinarySearch(info, item, first, midPoint-1);    // general case 1
        else if (item == info[midPoint]) {                        // base case 2
            item = info[midPoint];
            return true;
        }
        else
            return BinarySearch(info, item, midPoint+1, last);    // general case 2
    }
}

```

Q82. Why is a separate wrapper function used, instead of making RetrieveItem itself recursive?

Answer

The recursive routine needs the extra parameters first and last in order to describe the shrinking subrange, but the client should not have to supply them - the client only knows about the item being sought.

The wrapper keeps the public interface clean by supplying the initial values 0 and length-1 and translating the boolean result into the found parameter. The same pattern appears in InsertItem/Insert and DeleteItem/Delete, where the wrapper supplies listData.

Q83. Apply the Three-Question Verification Method to the recursive BinarySearch.

Answer

Base-Case Question: yes. There are two non-recursive exits - first > last returns false, and item == info[midPoint] returns true - and both are correct: an empty range cannot contain the item, and an equal element is the item.

Smaller-Caller Question: yes. Each general case excludes midPoint, passing either first..midPoint-1 or midPoint+1..last, so the range strictly shrinks every time and must eventually become empty or hit the item.

General-Case Question: yes. Because the array is sorted, if item < info[midPoint] then item can only lie in the lower half, and symmetrically for the upper half; so assuming the recursive call searches its half correctly, the whole function is correct.

Q84. Why do the general cases of BinarySearch use midPoint-1 and midPoint+1 rather than midPoint? What would go wrong otherwise?

Answer

info[midPoint] has already been compared and ruled out, so excluding it is safe.

More importantly it is what guarantees termination. If the call were BinarySearch(info, item, first, midPoint), then a two-element range with first = midPoint would regenerate the identical range on every call: the size factor would never decrease, the base case would never be reached, and the recursion would run until the run-time stack was exhausted.

Q85. For the recursive InsertItem on a sorted linked list, state the size factor, base cases and general case, and write the code.

Answer

Size factor: the number of elements in the current list. Base cases: (1) if the list is empty, insert the item into the empty list; (2) if item < location->info, insert the item at the front of the current list. General case: Insert(location->next, item).

```
template <class ItemType>
void SortedType<ItemType>::InsertItem(ItemType newItem)
{
    Insert(listData, newItem);
}

template <class ItemType>
void Insert(NodeType<ItemType>* &location, ItemType item)
{
    if ((location == NULL) || (item < location->info)) {    // base cases
        NodeType<ItemType>* tempPtr = location;
        location = new NodeType<ItemType>;
        location->info = item;
        location->next = tempPtr;
    }
    else
        Insert(location->next, item);                    // general case
}
```

Q86. Explain the meaning and the necessity of the parameter declaration NodeType<ItemType>* &location.

Answer

It is a reference to a pointer. location is not a copy of the caller's pointer - it is an alias for the actual pointer variable itself. At the top level it is an alias for listData; in the first recursive call it is an alias for N1->next; in the next, for N2->next; and so on.

This is essential because the insertion works by ASSIGNING to location. Writing location = new NodeType<ItemType>; must change the real link inside the previous node, not a local copy. Without the &, the new node would be attached to a temporary pointer and the list would be left unchanged - the node would simply leak.

Q87. Explain why no predLoc pointer is needed in the recursive insertion, whereas the non-recursive version requires one.

Answer

The iterative version walks the list with a single pointer, and once it has found the correct position it can no longer reach the node before it. It must therefore drag a trailing pointer predLoc along so that predLoc->next can be updated.

In the recursive version the reference parameter IS the predecessor's next field. The recursion carries the exact link that has to be modified all the way down, so the predecessor is never lost and no extra pointer is needed.

Q88. Explain the three statements in the base case of the recursive Insert, in order, and say what would break if they were reordered.

Answer

```
NodeType<ItemType>* tempPtr = location;    // 1. save the rest of the list
location = new NodeType<ItemType>;        // 2. put a new node in the gap
location->info = item;                     // 3. store the data
location->next = tempPtr;                   // 4. attach the saved remainder
```

Step 1 must come first because step 2 overwrites location. If new were called before saving, the address of the remainder of the list would be lost, the new node's next could not be set correctly, and every node after the insertion point would be leaked. This is the same save-then-overwrite discipline used by Push and Pop in the linked stack.

Q89. For the recursive DeleteItem on a sorted linked list, state the size factor, base case and general case, and write the code.

Answer

Size factor: the number of elements in the list. Base case: if item == location->info, delete the node pointed to by location. General case: Delete(location->next, item).

```
template <class ItemType>
void SortedType<ItemType>::DeleteItem(ItemType item)
{
    Delete(listData, item);
}

template <class ItemType>
void Delete(NodeType<ItemType>* &location, ItemType item)
{
    if (item == location->info) {
        NodeType<ItemType>* tempPtr = location;
        location = location->next;
        delete tempPtr;
    }
    else
        Delete(location->next, item);
}
```

Because location is an alias for the previous node's next field, the single assignment location = location->next both bypasses the node in the list and updates the real link. tempPtr keeps the address so the node can still be freed.

Q90. What assumption does the recursive Delete make, and how would you make it safe?

Answer

It assumes the item is definitely present in the list. There is no base case for location == NULL, so if the item is absent the recursion runs off the end and dereferences a NULL pointer.

```
if (location == NULL)
    return; // or throw / report not found
else if (item == location->info) { ... }
else Delete(location->next, item);
```

Adding that guard supplies a second base case and makes the Smaller-Caller Question answerable: the recursion now terminates on every input.

Q91. Given the condition that you must not use any loop, write a recursive function that returns the number of nodes in a linked list.

Answer

```
template <class ItemType>
int Length(NodeType<ItemType>* location)
{
    if (location == NULL) // base case
        return 0;
    else
        return 1 + Length(location->next); // general case
}
```

Size factor: the number of nodes from location onwards. Base case: an empty list has length 0. General case: the length is one (for the current node) plus the length of the rest. Each call moves one

node closer to NULL, so termination is guaranteed. Time is $O(N)$ and stack space is also $O(N)$, which is why an iterative version would be preferable for a long list.

Q92. Given the condition that you must not use a loop, write a recursive function that prints a linked list in REVERSE order.

Answer

```
template <class ItemType>
void PrintReverse(NodeType<ItemType>* location)
{
    if (location == NULL)
        return; // base case: nothing to print
    PrintReverse(location->next); // first print everything after
    cout << location->info << " "; // then print this node
}
```

Logic: the whole trick is that the output statement comes AFTER the recursive call. The calls descend to the end of the list without printing anything, and printing happens as the activation records are popped - so the last node is printed first. This is a direct use of the LIFO behaviour of the run-time stack, and is the same effect you would get by pushing every item onto a stack and then popping.

Q93. Given the condition that the list may be unsorted, write a recursive function that returns the sum of all items in a linked list of integers.

Answer

```
int Sum(NodeType<int>* location)
{
    if (location == NULL) // base case
        return 0;
    else
        return location->info + Sum(location->next);
}
```

Size factor: the number of nodes remaining. The base case returns the identity element for addition, 0, so an empty list contributes nothing. The general case combines the current value with the sum of the rest. Sortedness is irrelevant here because every node must be visited anyway, giving $O(N)$.